5:04AM
Friday
22/8/25

classmate
Date
Page 1

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

* Android Architecture

- Android architecture is the layered structure that defines how the Android OS is built, how components interact & how developers build apps on top of it. It is a stack of layers, where each layer provides services to the layer above it.

Android architecture consists of 5 main layers:

1) Applications (Apps Layer) — System & user apps
2) Java API framework
3) Native Libraries & Android Run Time (ART)
4) Hardware Abstraction Layer (HAL)
5) Linux Kernel

Applications (Contacts, Phone, Browser, Your App)

Java API framework (Activity Manager, Window Manager, etc.)

Native Libraries (C, C++, OpenGL, SSL, etc) | Android Runtime

Hardware abstraction Layer (HAL)

Linux Kernel (Drivers: Display, Camera, Bluetooth, Wifi)

1) Applications (Top Layer)

- This is where your Android apps live (both system apps & user installed)
- Example: Phone, contacts, Messages, settings.
- This is the layer developers actually build. The app runs on top of Java API framework, indirectly using all lower layers.

2) Java API framework

- This is the toolbox of APIs that app developers use.
- It exposes android system features through Java/Kotlin classes.
- Example: Activity manager manages app lifecycle.
  Window manager manages UI windows
  View System provides UI components like TextView, Button

3) Native Libraries & Android Runtime (ART)

(a) Native C/C++ Libraries: Core system libraries written in C/C++ like SQLite, WebKit, Media Framework

(b) Android RunTime: It executes app bytecode compiled from Java/Kotlin into DEX (Dalvik Executable) format.
Key functions are Ahead-of-Time (AOT), Just-In-Time (JIT) compilation, Garbage collection, Memory Management.

4) Hardware Abstraction Layer (HAL): It provides standard interface for hardware vendors to implement drivers. You don't code HAL directly, but when you use APIs like CameraX you are indirectly interacting with HAL.

5) Linux Kernel (Lowest Layer): Acts as foundation of Android OS. Provides hardware abstraction & low-level services. Responsible for process management, memory management, device drivers, power management. Developers rarely interact with this directly, but the apps ultimately rely on these kernel-level services.

* Activity in Android & how to create one

- An activity is a single screen with a user interface (UI) in an android app. Acts an an entry point for user interaction. Manages the UI components (views) & handles user events.

- **MainActivity.java**

```
package com.example.myfirstapp;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class SecondActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState)
    { super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_second);
    }
}
```

- activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:padding="16dp">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello from Second Activity!"
        android:textSize="20sp" />

</LinearLayout>
```

* Activity Lifecycle

- An activity in Android goes through different states. The Android OS manages these states by calling lifecycle callback methods.

7 most important callbacks:

1) **OnCreate(Bundle savedInstanceState)**: Called when the activity is first created. Used for initial setup. Always called once in the activity's lifetime.

2) onStart(): Activity becomes visible to the user, but not yet interactive.

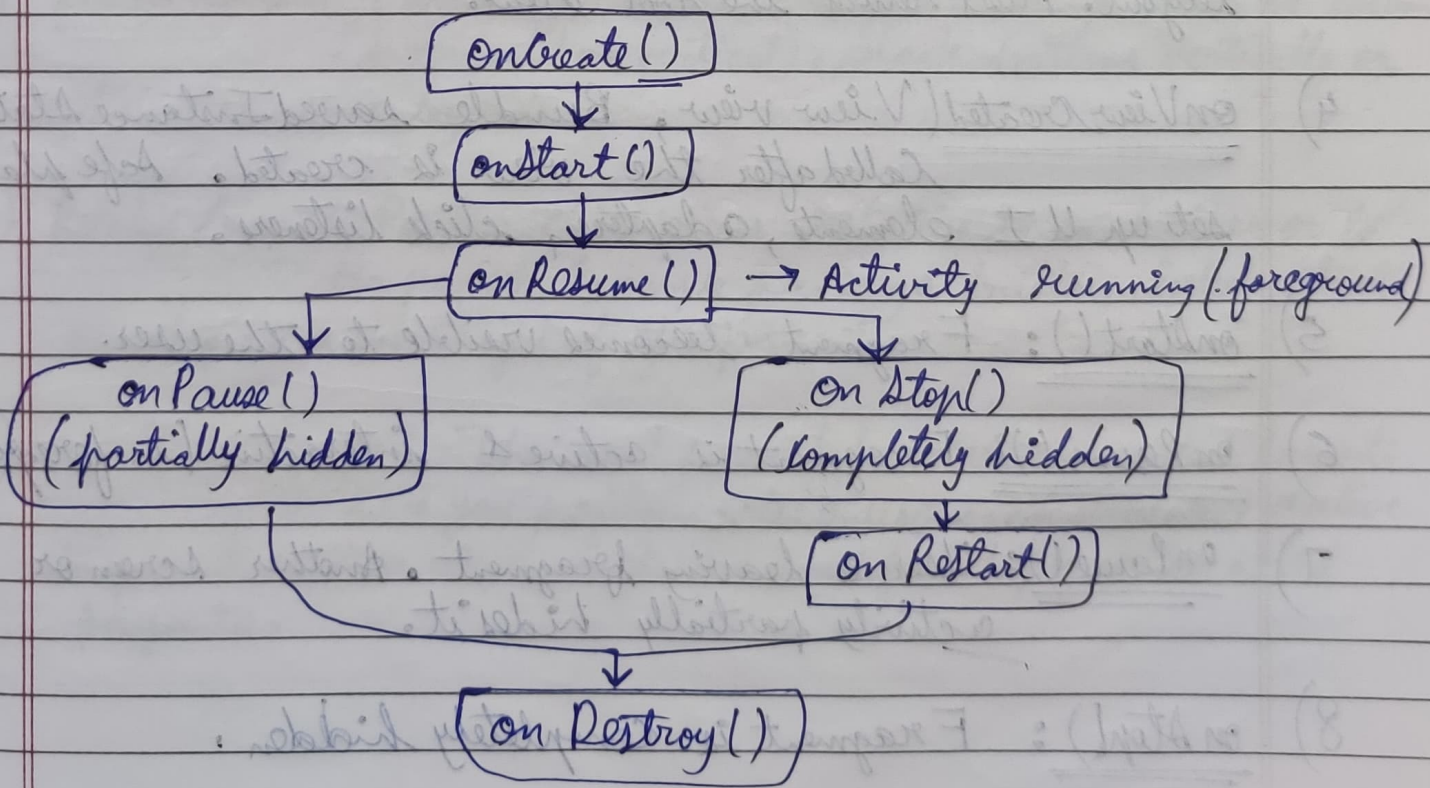
3) onResume(): Activity is now in the foreground & the user can interact with it.

4) onPause(): Called when activity is partially hidden. Use this to paus animations, release sensors, save temporary data.

5) onStop(): Activity is completely hidden. It frees up resources here.

6) onRestart(): Called just before onStart() if the activity is coming back to the foreground after being stopped

7) onDestroy(): Called when activity is finishing or the system is killing it. Use this to clean up resources.



* Fragment Lifecycle

- A fragment is like a mini-activity : it has its own UI layout, its own lifecycle, it always live inside an activity. Useful for building modular, reusable UI components.

- Fragment lifecycle methods :

1) onAttach(Context context): Fragment is attached to an activity. Best place to get reference to the hosting activity.

2) onCreate(Bundle savedInstanceState): Called to initialize fragment logic, not UI.

3) onCreateView(LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState): Called to inflate fragment's UI layout. Must return the root view.

4) onViewCreated(View view, Bundle savedInstanceState): Called after the view is created. Safe place to set up UI elements, adapters, click listeners.

5) onStart(): Fragment becomes visible to the user.

6) onResume(): Fragment is active & interactive (in foreground).

7) onPause(): User is leaving fragment. Another screen or activity partially hides it.

8) onStop(): Fragment is completely hidden.

9) onDestroyView() : Fragment's UI view is destroyed, but fragment instance still exists. Free UI resources here

10) onDestroy(): Final cleanup of fragment state.

11) onDetach(): Fragment is detached from the activity.

* Layouts

- A layout in Android defines the structure (arrangement) of UI elements (views like TextView, Button, EditText, ImageView) on the screen. Layouts are written in XML files (inside res/layout/). The layout acts as a container (ViewGroup) that decides how views will be arranged (position, size, alignment). child

1) Linear Layout : Arranges child views in a single row or column (horizontal or vertical). Stack buttons vertically or place them side by side.

2) Relative Layout: Positions child views relative to each other or to the parent. Example placing a button below a text or aligned to the right of the parent.

3) Constraint layout : The most powerful & recommended layout. Lets you position views using constraints relative to parent or other views. Reduced need for deeply ~~nested~~ nested layouts.

4) FrameLayout: A simple layout to stack views on top of each other (like layers). Example display an image & overlay text on top of it.

5) TableLayout: Arranges views in rows & columns like an HTML table. Example is a calculator app keypad.

6) GridLayout: Arranges child views in a grid pattern (rows × columns). Example gallery of images.

* Intent & its types

- Intent is an object used to request an action from another Android component. Its like a message-passing mechanism. With intents, we can start a new activity, start a service & deliver a broadcast.

Types of Intents:

1) Explicit Intent: You specify the exact target component you want to start. Used within our own app to navigate ~~foto~~ between activities.

Example: start SecondActivity from MainActivity

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivity(intent);
```

2) **Implicit Intent:** You don't specify the component, instead you declare a general action you want to perform. The Android system finds the best app/component to handle it. Used to open system apps or other apps.

Example: Open a webpage

```
Intent intent = new Intent( Intent.ACTION_VIEW);
intent.setData( Uri.parse("https://www.google.com"));
startActivity( intent);
```

→ This will open the browser app

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

19 September 2025 - Friday - 2:55PM

Q) What is CardView? Show XML to include CardView inside RecyclerView item layout.

Ans) CardView is a UI component provided by Android. Its a FrameLayout with rounded corners, shadow & a consistent card like look.

- It provides material design shadow & elevation out of the box.
- Supports corner radius & card background colour.
- Easy to make UI elements look clean & modern.

→ When we create a RecyclerView, each item in the list is usually inflated from an XML layout file. If we wrap the content of that item ~~for~~ tag inside a CardView, each item will look like a material card.

* Explanation of XML code:

→ CardView acts as a parent container with margin, rounded corners & shadow.
→ LinearLayout holds item content like TextView & ImageView
→ RecyclerView will use this item_layout.xml for each row, giving a card-styled list.

* XML code

```
<androidx.cardview.widget.CardView

    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:card_view="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    card_view:cardCornerRadius="8dp"
    card_view:cardElevation="4dp">
<LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp"
        android:orientation="vertical">

<TextView
        android:id="@+id/titleText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Title"
        android:textSize="18sp"
        android:textStyle="bold" />

<TextView
        android:id="@+id/subText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Subtitle" />
</LinearLayout>
</androidx.cardview.widget.CardView>
```

॥ जय माँ ॥

28 September 2025 — Sunday

Q12) XML code for simple Linear Layout with TextView & Button — 3M

Ans12)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello Android!"
    android:textSize="18sp" />

    <Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Click Me!" />

</LinearLayout>
```

Q13) Recycler View. How is it different from List View? — 3M

Ans13) Recycler View is an advanced & flexible view in Android used to display large sets of data in a list or grid format. It efficiently reuses ("recycles") item views that are off-screen, improving performance.

- View Holder pattern is built-in in Recycler View while optional in List View.
- Layout Manager in Recycler View allows different layouts while List View supports only vertical lists.
- Recycler View supports item animations & decorators easily, which List View lacks.

Q19) Design & explain a custom UI layout for a Login Screen — Username, Password, Login Button — 8M

Ans19)
```
<? xml version="1.0" encoding="utf-8" ?>
<Linear Layout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp" >
```

```
<EditText
    android:id="@+id/Username"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:hint="Username"
    android:inputType="textPersonName"
    android:marginBottom="12dp" />

<EditText
    android:id="@+id/Password"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Password"
    android:inputType="textPassword"
    android:marginBottom="16dp" />

<Button
    android:id="@+id/btnlogin"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Login"
    android:textColor="#FFFFFF" />

</LinearLayout>
```

Q24) Difference b/t" Relative Layout & Linear Layout — 3M

Ans 24)

Relative Layout	Linear Layout
• Arranges child views relative to each other or to the parent. Eg to Right Of, below.	Arranges child views in a single direction — either vertical or horizontal.
• More flexible for complex UIs.	Inefficient for complex UIs. i.e. less flexible.
• Heavier in performance.	Lighter in performance
• Supports overlapping views	Does not support overlapping views

—3M

Q26) XML code to arrange three buttons vertically using Linear Layout

Ans 26)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    >
<Button
android:layout_height="wrap_content"
android:layout_width="match_parent"
android:text="Button 1"/>
```

```
<Button
android:layout_height="wrap_content"
android:layout_width="match_parent"
android:text="Button 2" />

<Button
android:layout_height="wrap_content"
android:layout_width="match_parent"
android:text="Button 3" />

</LinearLayout>
```

Q16) XML & Java code to add a Toolbar to an Activity - 6M

Ans 16) activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical" >

<androidx.appcompat.widget.Toolbar
android:id="@+id/toolbar"
android:layout_width="match_parent"
android:layout_height="wrap_content"
app:title="My Toolbar"
app:titleTextColor="@android:color/white" />

</LinearLayout>
```

MainActivity.java

```
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        Toolbar toolbar = findViewById(R.id.toolbar);

        setSupportActionBar(toolbar); //sets Toolbar as
                                      //actionbar
    }
}
```

Q15) Explain with code how to create an Options Menu with 3 menu items — 4M

Ans 15)

menu_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu
    <item
        android:id="@+id/menu_item1"
        android:title="Item 1" />
```

```
<item
    android:id="@+id/menu_item2"
    android:title="Item2" />

<item
    android:id="@+id/menu_item3"
    android:title="Item3" />

</menu>
```

MainActivity.java

```
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState){

        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu){

        getMenuInflater().inflate(R.menu.menu_main,
                                  menu);
        return true;
    }
```

```
@Override
public boolean onOptionsItemSelected(MenuItem item){
String msg = " ";

switch (item.getItemId()) {

case R.id.menu_item1: msg = "Item1 clicked";
break;

case R.id.menu_item2: msg = "Item2 clicked";
break;

case R.id.menu_item3: msg = "Item 3 clicked";
break;

default: return super.onOptionsItemSelected(item);
}
Toast.makeText(this, msg, Toast.LENGTH_SHORT).
                    show();
return true;
}
}
```

* Unit-1

Q1) Code for Toast message in Android — 3M

Ans1)

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main_activity);
        Toast.makeText(this,
        "Hello, this is a toast Message!",
        Toast.LENGTH_SHORT).show();
    }
}
```

→ Toast message is a small popup message that provides simple feedback to the user.

→ It appears for a short duration & disappears automatically.

→ Examples like "Data Saved", "Login Successful".

Q2) What is the function of Android Manifest.xml? - 3M

Ans2) The Android Manifest.xml file is a mandatory configuration file in every Android app. It provides essential information about the app to the Android system before the app runs.

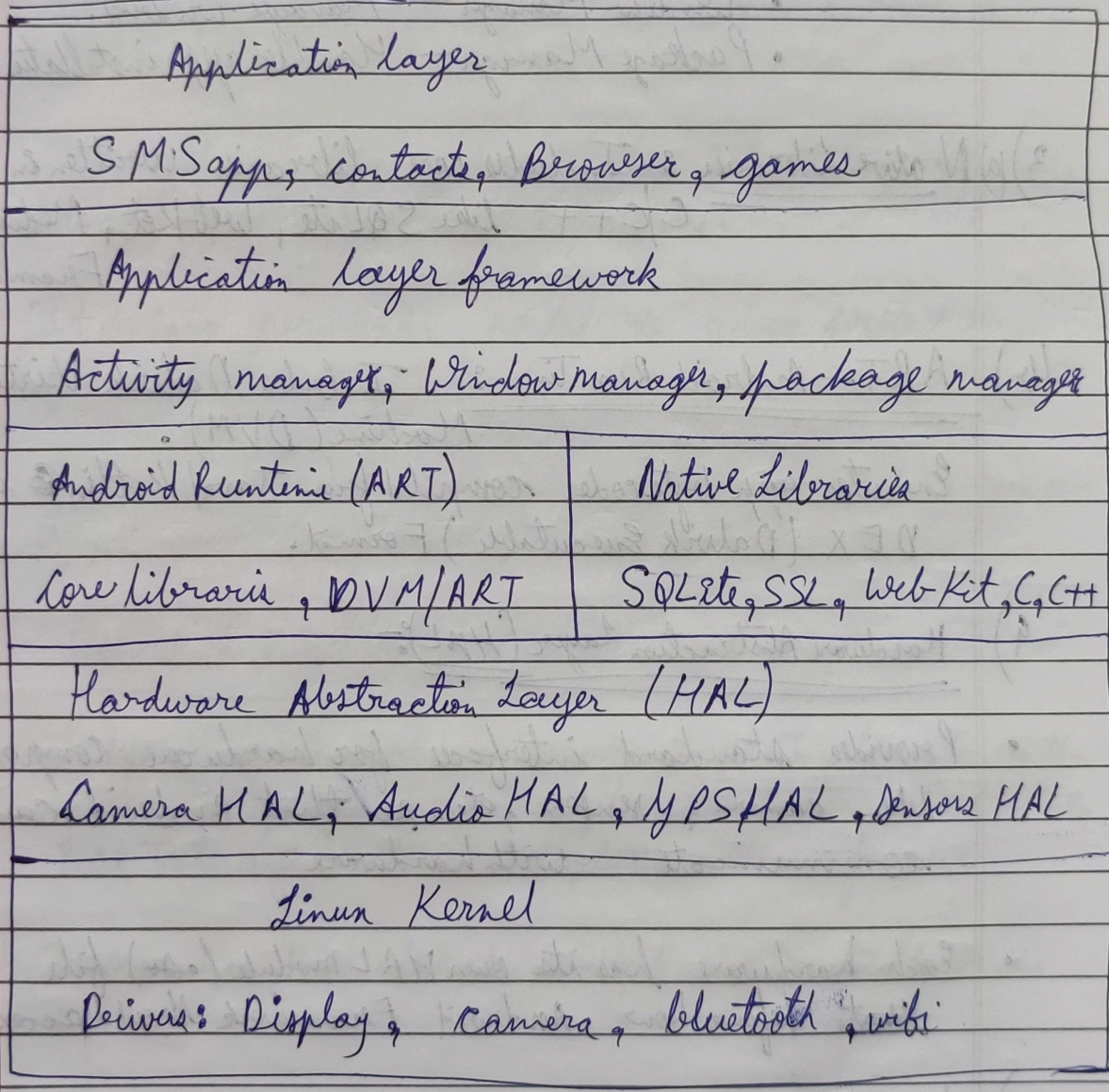
Key fns :-

1) Declares app components: Activities, Services, Content Providers.
2) Specifies the permission the app needs like Camera, Net.
3) Defines app metadata, such as app name, icon, theme.
4) Sets the minimum & target SDK versions.
5) Declares intent filters to indicate which activities can be launched by other apps or the system.

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

7 October 2025 — Tuesday

* Android Architecture



1) Applications Layer

- Topmost layer where users interact
- Contains both system apps & user-installed apps
- All apps are written in Java / Kotlin using Android SDK

2) Application Framework Layer

- Toolbox of APIs that developers use.
- Components
  - Activity Manager — Manages app life cycle
  - Window Manager — Manage windows.
  - Package Manager — Handles app installation.

3) (a) Native Libraries : Includes core libraries written in C/C++ like SQLite, WebKit, Media Framework

(b) ART — Android Run Time : Includes Dalvik Virtual Machine (DVM).
Executes app bytecode compiled from Java/Kotlin into DEX (Dalvik Executable) Format.

4) Hardware Abstraction Layer (HAL):-

- Provides standard interface for hardware components like camera, sensors, etc. so that android can communicate with hardware
- Each hardware has its own HAL module (.so) file that define how Android Framework should access it.

5) Linux Kernel: Core of the Android OS.
Provides hardware abstraction, memory management, process management, device drivers, security management. Examples : Display, Camera, Wifi, audio,

* Uses of Android Manifest File

- It acts as a blueprint or configuration file that tells Android System how to run & manage your app
- It declares app name, icon, version, package name & minimum/target SDK versions.
- It declares app components like Activities, Services, broadcast receivers, content providers.
- It declares permission needed to access protected feature like camera, internet, location
- It declares hardware/software features required to run our app.

* Layout & advantages

- It defines the structure & arrangement of UI elements on the screen. It specifies how widgets like Button, Text View, Edit Texts are positioned & displayed within a fragment.
- It organizes UI elements neatly & systematically on the screen
- It ensures app look good on different screen size & orientation
- Layouts are defined separately in XML, making code clean, reusable & easy to modify.
- Android provides multiple layouts like LinearLayout, Relativ offering flexibility in UI design.

* Code snippet for Toast Message

→ Toast.makeText(getApplicationContext(), "Hello, Android!", Toast.LENGTH_SHORT).show();

- makeText() → create toast message
- getApplicationContext() → provides context
- "Hello, Android!" → message to display
- Toast.LENGTH_SHORT → duration = short
- show() → displays toast on screen

→ Toast is a short popup message used to show feedback to user

* Custom Layout : creating our own user interface design instead of using the predefined layouts like Linear Layout or Relative Layout.

- It allows developers to combine existing views or create new view classes to achieve a unique UI.
- It offers better control over arrangement & behaviour of elements.
- We can combine multiple built-in layouts & make our own layout design like nesting Linear & Relative Layout

* **Grid Layout** – A layout manager in Android that arranges UI elements in a rectangular grid – made up of rows & columns.

- Each cell in the grid can ~~easily~~ contain one View (widget) & we can control the position & size of each view easily.

→
```
<GridLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:rowCount="2"
    android:columnCount="2">

    <Button android:text="A" />
    <Button android:text="B" />
    <Button android:text="C" />
    <Button android:text="D" />

</GridLayout>
```

* JSON - Java Script Object Notation is a lightweight data-interchange format used to store & exchange data between a server & a client.

It is text based, human-readable & uses key-value pairs like Javascript objects.

* Why JSON over XML

- Simpler syntax, small size, makes data transfer & parsing faster than XML.
- JSON is easier for humans to read & for programs to parse easily into objects.
- JSON is light weight, text based, simpler, faster.

*

Internal Storage	External Storage
• Memory area inside the device, private to app.	Memory area outside the app's sandbox eg SD card.
• Only app itself can access the data	Data can be accessed by other apps or users.
• More secure	Less secure
• Deleted when app is uninstalled	Remains even after app uninstalled
• Store private files, settings or cache.	Store media files, downloads or user-shared data.

* Web Service is a software application that allows communication bet" 2 devices over the internet using standard web protocols like HTTP, XML or JSON.

- It enables data exchange bet" different apps, regardless of platform or language.
- Example: An Android app using a weather API to get live weather data.

Web Service	Web Page
• Used for machine-to-machine communication (data exchange)	Used for human interaction i.e. information display
• Returns data (XML/JSON)	Returns HTML content
• Accessed by apps/programs	accessed by users via browsers

* Parsing is the process of reading, analyzing & converting data from one format like JSON/XML into a usable form within a program.

- It is used to extract specific information like name, email, IDs from data received from a web service or API.
- In Android, JSON parsing can be done by "org.json" package which provides classes like:
  1) JSONObject
  2) JSONArray

* Parsing using JSON Object

→ JSON Data
```
{ "name" : "Vasudev Krishna",
  "age" : 24,
  "city" : "Dwarka"
}
```

→

```
try { String jsonData = "{\"name\":\"Vasudev Krishna\",
                          \"age\":23,
                          \"city\":\"Pune\"}";
JSONObject jsonObject = new JSONObject(jsonData);
String name = jsonObject.getString("name");
int age = jsonObject.getInt("age");
String city = jsonObject.getString("city");
Toast.makeText(this, name + ", " + age + ", "
+ city, Toast.LENGTH_SHORT).show();
} catch (JSONException e){
    e.printStackTrace();
}
```

→ JSONObject parses single JSON objects
→ JSONArray parses array of objects.

Retrofit

- It is type safe HTTP client library for Android & Java, developed by Square.
- It is used to make network requests to web services & handle responses (JSON & XML).
- Simplifies API calls.
- Supports JSON parsing.
- Handles asynchronous & synchronous requests.
- Retrofit is a powerful & easy-to-use library for parsing API calls & making network requests in Android.

Fragment	Activity
A modular part of an activity	A single screen in an app
Represents a portion of UI	Interacts with UI
Lifecycle dependent on host activity	Has its own lifecycle.
Reusable in multiple activities & layouts.	Cannot be reused directly, new activity needs to be created.
Cannot exist independently must exist in an activity	Can exists independently.

Q) What is parsing? How can you perform parsing using JSON in Android Application — 8 M

Ans) Parsing is the process of reading, analyzing and converting data from one format into a usable form for a program.

- In Android, parsing is mainly used to extract meaningful information from structured data formats such as JSON, XML, etc.

- Example: If an app recieves data from a server in JSON format, parsing helps to break the data into objects, strings, numbers, etc so the app can display it properly.

* JSON Parsing in Android

- JSON (JavaScript Object Notation) is a lightweight data format commonly used for data exchange in Android Apps — especially in APIs & web services.

- Android provides multiple ways to parse JSON:-

1) JSON Object / JSON Array (built-in parser).
2) GSON — Google Library
3) Moshi / Jackson (external libraries)

* Steps to perform JSON parsing using JSON Object/ JSON Array

1) Recieve JSON Data :- Data may come from a server, file or local string.

2) Create a JSON Object :- Using JSON Object to read key-value pairs.

3) Extract Values :- Using methods like getString(), getInt(), getJSONArray(), etc.

4) Use data inside the app :- Display in TextView, ListView, RecyclerView, etc.

* Example Program : JSON Parsing in Android (Java)

```
{
    "name" : "John",
    "age" : 25,
    "city" : "Mumbai"
}
```

Q) Write a Retrofit Interface to get a list of users from a sample API endpoint — 4M.

Ans)
```
import java.util.List;
import retrofit2.Call;
import retrofit2.http.GET;
public interface ApiService{
    @GET("users")
    Call<List<User>> getUsers();
}
```

Q) 3 Components ~~to~~ required to make a Retrofit network call.-3M

Ans) ① Retrofit Instance — Configured with a base URL & a converter

② API Interface — Contains endpoints using annotations like @GET, @POST.

③ Model/POJO Classes — Used to map the JSON response into Java objects.

Unit-1

Q1) Code snippet for toast message — 3M

Ans1) Toast.makeText(this, "Hello, this is toast message", Toast.LENGTH_SHORT).show();

Q6) Activity lifecycle code for each state — 8M

Ans6) 1) onCreate() — Activity launched

2) onStart() — Activity becomes visible to user

3) onResume() — Enters foreground

4) onPause() — Partially visible

5) onStop() — No longer visible

6) onRestart() — Coming back to foreground

7) onDestroy() — " Destroyed.

```
@Override
protected void onCreate(Bundle b){
    Super.onCreate(b);
    Set Content View(R.layout.activity_main);
    Log.d("Lifecycle", "onCreate");
}
```

```
@Override
protected void onX() {
    super.onX();
    Log.d("Lifecycle", "X");
}
```

X = onStart(), onResume(), onPause(), onStop(), onRestart(), onDestroy().

Unit-2

Q15) Code OptionsMenu with 3 Menu items —4M

Ans15)

XML

```
<menu
  <item
    android:id="@+id/m1"
    android:title="Home" />

  <item
    android:id="@+id/m2"
    android:title="Settings" />

  <item
    android:id="@+id/m3"
    android:title="Logout" />
</menu>
```

Java

```
public class MainActivity extends AppCompatActivity{

  @Override

  protected void onCreate(Bundle b){

      super.onCreate(b);

      setContentView(R.layout.activity_main);

  }
```

```
@ Override
public boolean onCreate OptionsMenu(Menu menu){
    getMenuInflater().inflate(R.menu.main_menu,
                              menu);
    return true;
}
@ Override
public boolean onOptionsItemSelected(MenuItem item)
    switch( item.getItemId()){
        case R.id.m1;    break;
        case R.id.m2;    break;
        case R.id.m3;    break;
    }
    return true;
}
```

Q16) add Toolbar to an activity — 4M

Ans16) <LinearLayout

XML

```
android:orientation = "vertical"
android:layout_width = "match_parent"
android:layout_height = "match_parent"

<androidx.appcompat.widget.Toolbar

android:id = "@+id/myToolbar"
android:layout_width = "match_parent"
android:layout_height = "?attr/actionBarSize"
android:background = "@color/teal_700" />

</LinearLayout>
```

Java

```
public class MainActivity Extends AppCompatActivity {

@Override
protected void onCreate (Bundle b) {

super.onCreate (b);

SetContentView (R.layout.activity_main);

Toolbar tb = findViewById (R.id.myToolbar);

SetSupportActionBar (tb);

} }
```

Unit-3

96)

getFilesDir()	getCacheDir()
• Returns the directory for internal storage files.	Returns the directory for internal cache files.
• Data stored here is persistent & not deleted automatically.	Data is temporary & may be deleted by the system when storage is low.
• Suitable for important user data, settings or files needed long term.	Used for images, temp resources, disposable data.

→ getFilesDir() = permanent storage

→ getCacheDir() = temporary storage (cache)

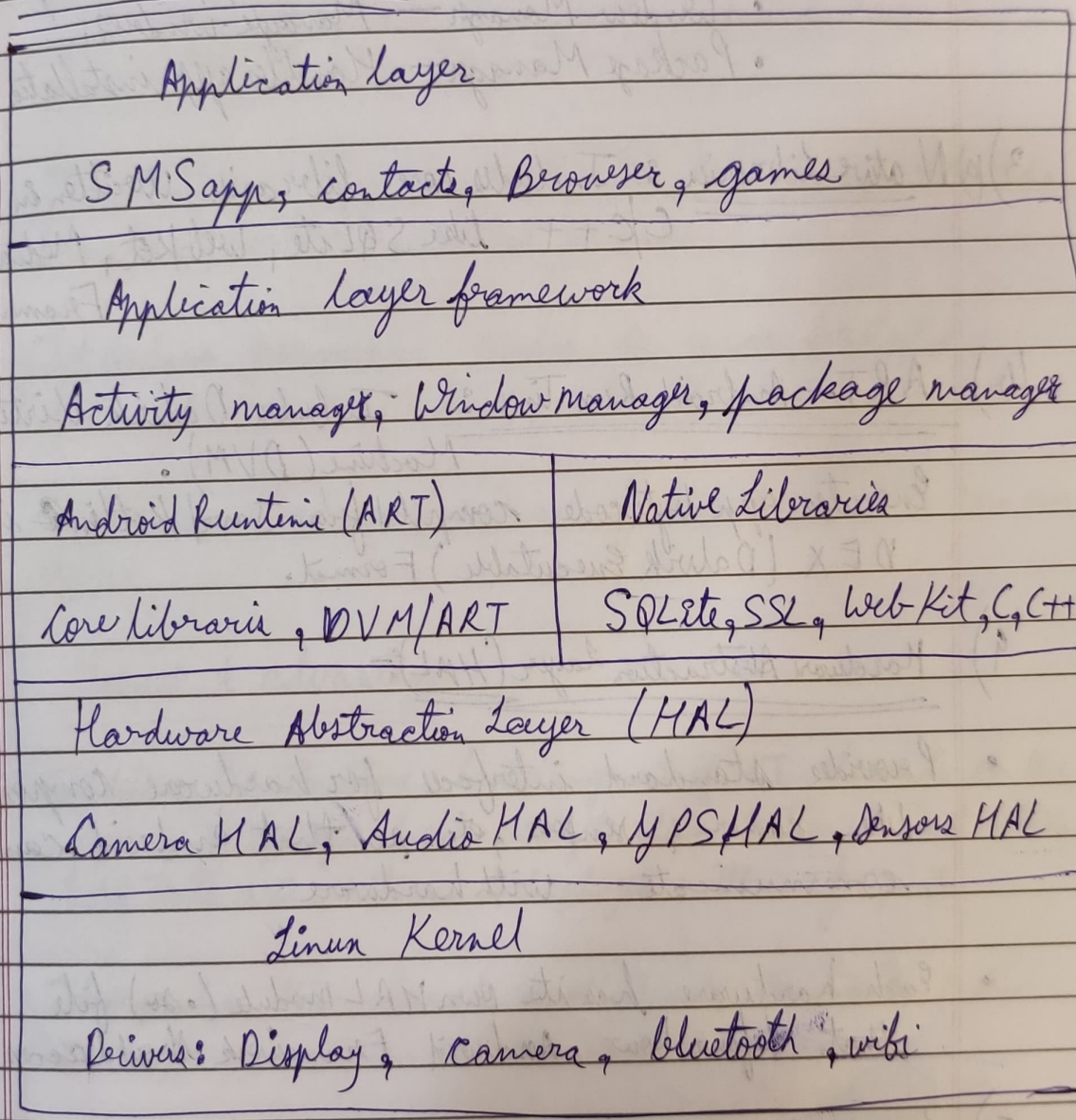
Q9) Code to create a folder in External storage — 3M

```
File folder = new File(Environment.
    getExternalStorageDirectory(), "MyFolder");
if (!folder.exists()){
    folder.mkdirs();
}
```

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

7 October 2025 — Tuesday

* Android Architecture



1) Application Layer

- Topmost layer where users interact
- Contains both system apps & user-installed apps
- All apps are written in Java / Kotlin using Android SDK

2) Application Framework Layer

- Toolbox of APIs that developers use.
- Components
  - Activity Manager — Manages app life cycle
  - Window Manager — Manage windows.
  - Package Manager — Handles app installation.

3)(a) Native Libraries : Includes core libraries written in C/C++ like SQLite, WebKit, Media Framework

(b) ART — Android Run Time : Includes Dalvik Virtual Machine (DVM).
Executes app bytecode compiled from Java/Kotlin into DEX (Dalvik Executable) Format.

4) Hardware Abstraction Layer (HAL):-

- Provides standard interface for hardware components like camera, sensors, etc. so that android can communicate with hardware
- Each hardware has its own HAL module (.so) file that defines how Android Framework should access it.

5) Linux Kernel : Core of the Android OS.
Provides hardware abstraction, memory management, process management, device drivers, security management. Examples : Display, Camera, Wifi, audio,

* Code snippet for Toast Message

→ Toast.makeText(getApplicationContext(), "Hello, Android!", Toast.LENGTH_SHORT).show();

- makeText() → create toast message
- getApplicationContext() → provides context
- "Hello, Android!" → message to display
- Toast.LENGTH_SHORT → duration = short
- show() → displays toast on screen

→ Toast is a short popup message used to show feedback to user.

* **Custom Layout** : creating our own user interface design instead of using the predefined layouts like Linear Layout or Relative Layout.

- It allows developers to combine existing views or create new view classes to achieve a unique UI.
- It offers better control over arrangement & behaviour of elements.
- We can combine multiple built-in layouts & make our own layout design like nesting Linear & Relative Layout

* JSON - Java Script Object Notation is a lightweight data-interchange format used to store & exchange data between a server & a client.

It is text based, human-readable & uses key-value pairs like Javascript objects.

* Why JSON over XML

- Simpler syntax, small size, makes data transfer & parsing faster than XML.
- JSON is easier for humans to read & for programs to parse easily into objects.
- JSON is light weight, text based, simpler, faster.

*	Internal Storage	External Storage
•	Memory area inside the device, private to app.	Memory area outside the app's sandbox eg SD card.
•	Only app itself can access the data	Data can be accessed by other apps or users.
•	More secure	Less secure
•	Deleted when app is uninstalled	Remains even after app uninstalled
•	Store private files, settings or cache.	Store media files, downloads or user-shared data.

* JSON - Java Script Object Notation is a lightweight data-interchange format used to store & exchange data between a server & a client.

It is text based, human-readable & uses key-value pairs like Javascript objects.

* Why JSON over XML

- Simpler syntax, small size, makes data transfer & parsing faster than XML.
- JSON is easier for humans to read & for programs to parse easily into objects.
- JSON is light weight, text based, simpler, faster.

*	Internal Storage	External Storage
•	Memory area inside the device, private to app.	Memory area outside the app's sandbox eg SD card.
•	Only app itself can access the data	Data can be accessed by other apps or users.
•	More secure	Less secure
•	Deleted when app is uninstalled	Remains even after app uninstalled
•	Store private files, settings or cache.	Store media files, downloads or user-shared data.

* Web Service is a software application that allows communication bet^n 2 devices over the internet using standard web protocols like HTTP, XML or JSON.

- It enables data exchange bet^n different apps, regardless of platform or language.
- Example: An Android app using a weather API to get live weather data.

Web Service	Web Page
• Used for machine-to-machine communication (data exchange)	Used for human interaction i.e. information display.
• Returns data (XML/JSON)	Returns HTML content
• Accessed by apps/programs	accessed by users via browsers

* Parsing is the process of reading, analyzing & converting data from one format like JSON/XML into a usable form within a program.

- It is used to extract specific information like name, email, IDs from data received from a web service or API
- In Android, JSON parsing can be done by "org.json" package which provides classes like:
  1) JSONObject
  2) JSONArray

* Retrofit

- It is type safe HTTP client library for Android & Java, developed by Square.
- It is used to make network requests to web services & handle responses (JSON & XML).
- Simplifies API calls.
- Supports JSON parsing.
- Handles asynchronous & synchronous requests.
- Retrofit is a powerful & easy-to-use library for parsing API calls & making network requests in Android.

*

Fragment	Activity
A modular part of an activity	A single screen in an app
Represents a portion of UI	Interacts with UI
Lifecycle dependent on host activity	Has its own lifecycle.
Reusable in multiple activities & layouts.	Cannot be reused directly, new activity needs to be created.
Cannot exist independently must exist in an activity	Can exists independently.